

**Московский государственный технический
университет им. Н. Э. Баумана**

Факультет «Информатика и системы управления»
Кафедра ИУ5 «Системы обработки информации и
управления»

Курс «Парадигмы и конструкции языков программирования»
Отчет по лабораторной работе №6

Выполнил:
Студент группы ИУ5-31Б
Блинкова Ксения

Проверил: Гапанюк Ю. Е.

2025 г.

Задание:

Разработать программу, реализующую работу с коллекциями.

1. Программа должна быть разработана в виде консольного приложения на языке C#.
2. Создать объекты классов «Прямоугольник», «Квадрат», «Круг».
3. Для реализации возможности сортировки геометрических фигур для класса «Геометрическая фигура» добавить реализацию интерфейса `Comparable`. Сортировка производится по площади фигуры.
4. Создать коллекцию класса `ArrayList`. Сохранить объекты в коллекцию. Отсортировать коллекцию. Вывести в цикле содержимое коллекции.
5. Создать коллекцию класса `List<Figure>`. Сохранить объекты в коллекцию. Отсортировать коллекцию. Вывести в цикле содержимое коллекции.
6. Модифицировать класс разреженной матрицы (проект `SparseMatrix`) для работы с тремя измерениями – x, y, z . Вывод элементов в методе `ToString()` осуществлять в том виде, который Вы считаете наиболее удобным. Разработать пример использования разреженной матрицы для геометрических фигур.
7. Реализовать класс «`SimpleStack`» на основе односвязного списка. Класс `SimpleStack` наследуется от класса `SimpleList` (проект `SimpleListProject`). Необходимо добавить в класс методы:
 - `public void Push(T element)` – добавление в стек;
 - `public T Pop()` – чтение с удалением из стека.
8. Пример работы класса `SimpleStack` реализовать на основе Геометрических фигур.

Листинг программы:

GeometricFigure.cs:

```
using System;

namespace GeometryApp
{
    public abstract class GeometricFigure : Comparable<GeometricFigure>
    {
        public abstract double Area();

        public int CompareTo(GeometricFigure? other)
        {
            if (other == null) return 1;
            return this.Area().CompareTo(other.Area());
        }

        public override string ToString()
        {
            return string.Format("Площадь = {0:F2}", Area());
        }
    }
}
```

```

        public virtual void Print()
        {
            Console.WriteLine(this.ToString());
        }
    }
}

```

IPrint.cs:

```

namespace GeometryApp
{
    public interface IPrint
    {
        void Print();
    }
}

```

Circle.cs:

```

using System;

namespace GeometryApp
{
    public class Circle : GeometricFigure
    {
        public double Radius { get; set; }

        public Circle(double radius)
        {
            Radius = radius;
        }

        public override double Area()
        {
            return Math.PI * Radius * Radius;
        }

        public override string ToString()
        {
            return string.Format("Круг (Радиус {0}), {1}", Radius,
base.ToString());
        }
    }
}

```

Rectangle.cs:

```
using System;

namespace GeometryApp
{
    public class Rectangle : GeometricFigure
    {
        public double Width { get; set; }
        public double Height { get; set; }

        public Rectangle(double width, double height)
        {
            Width = width;
            Height = height;
        }

        public override double Area()
        {
            return Width * Height;
        }

        public override string ToString()
        {
            return string.Format("Прямоугольник (Ш {0}, В {1}), {2}", Width,
Height, base.ToString());
        }
    }
}
```

Square.cs:

```
using System;

namespace GeometryApp
{
    public class Square : GeometricFigure
    {
        public double Side { get; set; }

        public Square(double side)
        {
            Side = side;
        }

        public override double Area()
        {
            return Side * Side;
        }
    }
}
```

```

        return Side * Side;
    }

    public override string ToString()
    {
        return string.Format("Квадрат (Сторона {0}), {1}", Side,
base.ToString());
    }
}

```

SimpleList.cs

```

using System;

namespace GeometryApp
{
    public class SimpleList<T> where T : IComparable<T>
    {
        protected class Node
        {
            public T Data;
            public Node? Next;
            public Node(T data) { Data = data; Next = null; }
        }

        protected Node? head;
        public int Count { get; protected set; }

        public virtual void Add(T element)
        {
            Node newNode = new Node(element);
            newNode.Next = head;
            head = newNode;
            Count++;
        }

        public void Sort()
        {
            if (head == null || head.Next == null) return;

            bool swapped;
            do
            {
                swapped = false;
                Node? current = head;
                Node? previous = null;

```

```

        while (current != null && current.Next != null)
        {
            if (current.Data.CompareTo(current.Next.Data) > 0)
            {
                T temp = current.Data;
                current.Data = current.Next.Data;
                current.Next.Data = temp;
                swapped = true;
            }
            previous = current;
            current = current.Next;
        }
    } while (swapped);
}

public override string ToString()
{
    Node? current = head;
    string result = "";
    while (current != null)
    {
        result += current.Data + Environment.NewLine;
        current = current.Next;
    }
    return result;
}
}
}

```

SimpleStack.cs

```

using System;

namespace GeometryApp
{
    public class SimpleStack<T> : SimpleList<T> where T : IComparable<T>
    {
        public void Push(T element)
        {
            Add(element);
        }

        public T Pop()
        {
            if (head == null) throw new InvalidOperationException("Стек пуст");
            T value = head!.Data;
            head = head!.Next;
            Count--;
        }
    }
}

```

```

        return value;
    }

    public T Peek()
    {
        if (head == null) throw new InvalidOperationException("Стек пуст");
        return head!.Data;
    }

    public bool IsEmpty()
    {
        return head == null;
    }
}

```

SparseMatrix.cs

```

using System;
using System.Text;
using System.Collections.Generic;

namespace GeometryApp
{
    public class SparseMatrix<T> where T : class
    {
        private Dictionary<string, T> matrix = new Dictionary<string, T>();
        private int sizeX, sizeY, sizeZ;

        public SparseMatrix(int x, int y, int z)
        {
            sizeX = x;
            sizeY = y;
            sizeZ = z;
        }

        public T? this[int x, int y, int z]
        {
            get
            {
                string key = GetKey(x, y, z);
                return matrix.ContainsKey(key) ? matrix[key] : null;
            }
            set
            {
                if (x < 0 || x >= sizeX || y < 0 || y >= sizeY || z < 0 || z >=
sizeZ)

```

```

        throw new IndexOutOfRangeException("Индекс выходит за границы
матрицы");

        string key = GetKey(x, y, z);
        if (value == null)
        {
            if (matrix.ContainsKey(key))
                matrix.Remove(key);
        }
        else
        {
            matrix[key] = value;
        }
    }

    private string GetKey(int x, int y, int z)
    {
        return $"{x},{y},{z}";
    }

    public override string ToString()
    {
        if (matrix.Count == 0)
            return "Матрица пуста";

        StringBuilder sb = new StringBuilder();
        sb.AppendLine($"Разреженная матрица {sizeX}x{sizeY}x{sizeZ}:");
        sb.AppendLine("Заполненные ячейки:");

        foreach (var kvp in matrix)
        {
            var coords = kvp.Key.Split(',');
            sb.AppendLine($" [{coords[0]},{coords[1]},{coords[2]}] =
{kvp.Value}");
        }

        sb.AppendLine($"Всего заполнено ячеек: {matrix.Count} из {sizeX * sizeY
* sizeZ}");
        return sb.ToString();
    }

    public void PrintALLLayers()
    {
        Console.WriteLine("Построчный вывод матрицы по слоям Z:");
        for (int z = 0; z < sizeZ; z++)
        {
            Console.WriteLine($" \nСлой Z = {z}:");
            for (int y = 0; y < sizeY; y++)
            {
                for (int x = 0; x < sizeX; x++)

```


Program.cs:

```
using System;
using System.Collections;
using System.Collections.Generic;

namespace GeometryApp
{
    class Program
    {
        static void Main()
        {
            Console.WriteLine("Лабораторная работа №6 - Работа с коллекциями");

            // ArrayList
            Console.WriteLine("Работа с ArrayList:");
            ArrayList arrayList = new ArrayList();
            arrayList.Add(new Rectangle(5, 3));
            arrayList.Add(new Square(4));
            arrayList.Add(new Circle(2.5));
            arrayList.Add(new Rectangle(2, 6));
            arrayList.Add(new Circle(1.8));

            Console.WriteLine("Содержимое ArrayList (без сортировки):");
            foreach (GeometricFigure fig in arrayList)
            {
                Console.WriteLine(fig);
            }

            // List<GeometricFigure>
            Console.WriteLine("\nРабота с List<GeometricFigure>:");
            List<GeometricFigure> list = new List<GeometricFigure>();
            list.Add(new Rectangle(5, 3));
            list.Add(new Square(4));
            list.Add(new Circle(2.5));
            list.Add(new Rectangle(2, 6));
            list.Add(new Circle(1.8));
        }
    }
}
```

```
Console.WriteLine("До сортировки:");
foreach (var fig in list)
{
    Console.WriteLine(fig);
}

list.Sort();

Console.WriteLine("\nПосле сортировки:");
foreach (var fig in list)
{
    Console.WriteLine(fig);
}

// SimpleStack
Console.WriteLine("Работа со стеком:");
SimpleStack<GeometricFigure> stack = new
SimpleStack<GeometricFigure>();

stack.Push(new Rectangle(4, 5));
stack.Push(new Square(3));
stack.Push(new Circle(2));
stack.Push(new Rectangle(6, 2));

Console.WriteLine("Содержимое стека после добавления элементов:");
Console.WriteLine(stack);

Console.WriteLine("\nИзвлечение элементов из стека:");
while (!stack.IsEmpty())
{
    GeometricFigure figure = stack.Pop();
    Console.WriteLine("Извлечен: " + figure);
}

//Демонстрация сортировки в SimpleList
Console.WriteLine("\n Демонстрация сортировки в SimpleList:");
SimpleList<GeometricFigure> simpleList = new
SimpleList<GeometricFigure>();
simpleList.Add(new Rectangle(4, 5));
simpleList.Add(new Square(3));
simpleList.Add(new Circle(2));
simpleList.Add(new Rectangle(6, 2));

Console.WriteLine("До сортировки:");
Console.WriteLine(simpleList);

simpleList.Sort();

Console.WriteLine("После сортировки:");
Console.WriteLine(simpleList);
```

```

// Работа с разреженной матрицей
Console.WriteLine("\nРабота с трехмерной разреженной матрицей:");

// Создаем разреженную матрицу 3x3x3
SparseMatrix<GeometricFigure> sparseMatrix = new
SparseMatrix<GeometricFigure>(3, 3, 3);

// Заполняем некоторые ячейки геометрическими фигурами
sparseMatrix[0, 0, 0] = new Rectangle(4, 5);
sparseMatrix[1, 1, 1] = new Square(3);
sparseMatrix[2, 0, 2] = new Circle(2.5);
sparseMatrix[0, 2, 1] = new Rectangle(6, 2);

// Выводим информацию о матрице
Console.WriteLine(sparseMatrix.ToString());

// Пример доступа к элементам
Console.WriteLine("\nПример доступа к элементам:");
Console.WriteLine($"Элемент [0,0,0]: {sparseMatrix[0, 0, 0]}");
Console.WriteLine($"Элемент [1,1,1]: {sparseMatrix[1, 1, 1]}");
var element001 = sparseMatrix[0, 0, 1];
Console.WriteLine($"Элемент [0,0,1] (пустая ячейка): {(element001 !=
null ? element001.ToString() : "null")}");

// Графическое представление матрицы
Console.WriteLine("\nГрафическое представление (X - заполненная
ячейка):");
sparseMatrix.PrintAllLayers();

// Демонстрация изменения элементов
Console.WriteLine("\nИзменение элементов матрицы:");
Console.WriteLine("Добавляем новый элемент в [1,0,0]");
sparseMatrix[1, 0, 0] = new Circle(1.8);
Console.WriteLine($"Теперь элемент [1,0,0] = {sparseMatrix[1, 0, 0]}");

Console.WriteLine("\nУдаляем элемент [2,0,2]");
sparseMatrix[2, 0, 2] = null;
var element202 = sparseMatrix[2, 0, 2];
Console.WriteLine($"Теперь элемент [2,0,2] = {(element202 != null ?
element202.ToString() : "null")}");

Console.WriteLine("\nОбновленная матрица:");
Console.WriteLine(sparseMatrix.ToString());
Console.WriteLine("\nНажмите любую клавишу для выхода...");
Console.ReadKey();
}
}
}

```

Результат выполнения:

```
ksu@LAPTOP-045GK84C:~/Blinkova_Labs_2025/dotnet$ dotnet run
Лабораторная работа №6 - Работа с коллекциями
Работа с ArrayList:
Содержимое ArrayList (без сортировки):
Прямоугольник (Ш 5, В 3), Площадь = 15.00
Квадрат (Сторона 4), Площадь = 16.00
Круг (Радиус 2.5), Площадь = 19.63
Прямоугольник (Ш 2, В 6), Площадь = 12.00
Круг (Радиус 1.8), Площадь = 10.18

Работа с List<GeometricFigure>:
До сортировки:
Прямоугольник (Ш 5, В 3), Площадь = 15.00
Квадрат (Сторона 4), Площадь = 16.00
Круг (Радиус 2.5), Площадь = 19.63
Прямоугольник (Ш 2, В 6), Площадь = 12.00
Круг (Радиус 1.8), Площадь = 10.18

После сортировки:
Круг (Радиус 1.8), Площадь = 10.18
Прямоугольник (Ш 2, В 6), Площадь = 12.00
Прямоугольник (Ш 5, В 3), Площадь = 15.00
Квадрат (Сторона 4), Площадь = 16.00
Круг (Радиус 2.5), Площадь = 19.63
Работа со стеком:
Содержимое стека после добавления элементов:
Прямоугольник (Ш 6, В 2), Площадь = 12.00
Круг (Радиус 2), Площадь = 12.57
Квадрат (Сторона 3), Площадь = 9.00
Прямоугольник (Ш 4, В 5), Площадь = 20.00

Извлечение элементов из стека:
Извлечен: Прямоугольник (Ш 6, В 2), Площадь = 12.00
Извлечен: Круг (Радиус 2), Площадь = 12.57
Извлечен: Квадрат (Сторона 3), Площадь = 9.00
Извлечен: Прямоугольник (Ш 4, В 5), Площадь = 20.00

Демонстрация сортировки в SimpleList:
До сортировки:
Прямоугольник (Ш 6, В 2), Площадь = 12.00
Круг (Радиус 2), Площадь = 12.57
Квадрат (Сторона 3), Площадь = 9.00
Прямоугольник (Ш 4, В 5), Площадь = 20.00

После сортировки:
Квадрат (Сторона 3), Площадь = 9.00
Прямоугольник (Ш 6, В 2), Площадь = 12.00
Круг (Радиус 2), Площадь = 12.57
Прямоугольник (Ш 4, В 5), Площадь = 20.00
```

Работа с трехмерной разреженной матрицей:

Разреженная матрица 3x3x3:

Заполненные ячейки:

[0,0,0] = Прямоугольник (Ш 4, В 5), Площадь = 20.00

[1,1,1] = Квадрат (Сторона 3), Площадь = 9.00

[2,0,2] = Круг (Радиус 2.5), Площадь = 19.63

[0,2,1] = Прямоугольник (Ш 6, В 2), Площадь = 12.00

Всего заполнено ячеек: 4 из 27

Пример доступа к элементам:

Элемент [0,0,0]: Прямоугольник (Ш 4, В 5), Площадь = 20.00

Элемент [1,1,1]: Квадрат (Сторона 3), Площадь = 9.00

Элемент [0,0,1] (пустая ячейка): null

Графическое представление (X - заполненная ячейка):

Построчный вывод матрицы по слоям Z:

Слой Z = 0:

[X][][]

[][][]

[][][]

Слой Z = 1:

[][][]

[][X][]

[X][][]

Слой Z = 2:

[][][X]

[][][]

[][][]

Изменение элементов матрицы:

Добавляем новый элемент в [1,0,0]

Теперь элемент [1,0,0] = Круг (Радиус 1.8), Площадь = 10.18

Удаляем элемент [2,0,2]

Теперь элемент [2,0,2] = null

Обновленная матрица:

Разреженная матрица 3x3x3:

Заполненные ячейки:

[0,0,0] = Прямоугольник (Ш 4, В 5), Площадь = 20.00

[1,1,1] = Квадрат (Сторона 3), Площадь = 9.00

[0,2,1] = Прямоугольник (Ш 6, В 2), Площадь = 12.00

[1,0,0] = Круг (Радиус 1.8), Площадь = 10.18

Всего заполнено ячеек: 4 из 27

Нажмите любую клавишу для выхода...

ksu@LAPTOP-04SGKB4C:~/Blinkova_Labs_2025/dotnet\$